Initiation à Python

Unité d'Enseignement UEF 2.1

1. Présentation de Python

Site du Cours:

eLearning

https://zair-bouzidi.github.io/zair-site/

Objectifs du cours

- À la fin de ce cours, l'étudiant doit être capable de :
  - comprendre ce qu'est Python ;
  - installer et lancer Python ;
  - écrire et exécuter un programme simple ;
  - utiliser les variables et les types de données ;
  - effectuer des opérations et des calculs ;
  - utiliser les structures conditionnelles ;
  - utiliser les boucles ;
  - manipuler les listes, tuples et dictionnaires ;

Python

- C'est un outil informatique créé par **Guido van Rossum en 1991**. Il permet d'écrire des instructions que l'ordinateur comprend et exécute.
- Il sert à créer des sites web, automatiser des tâches, analyser des données et faire de l'intelligence artificielle.
- Sa lecture est simple et facile pour les débutants. Son nom vient de la **troupe comique britannique des *Monty Python***, et non du reptile (grand serpent non venimeux qui étouffe ses proies (un constricteur), vivant dans les régions chaudes).

Objectifs du cours

- À la fin de ce cours, l'étudiant doit être capable de :
  - créer et utiliser des fonctions ;
  - lire les données saisies par l'utilisateur ;
  - lire et écrire des fichiers ;
  - comprendre les notions élémentaires de modules et bibliothèques ;
  - réaliser un petit programme Python.

2. Qu'est-ce que Python ?

Python

- **Python** est un langage de programmation :
  - libre et open source ;
  - gratuit ;
  - multiplateforme : Windows, Linux, macOS ;
  - relativement facile à apprendre ;
  - interprété ;
  - de haut niveau ;
  - largement utilisé en IA, data science, développement Web, automatisation, etc.

Exemple :

- **Exemple :**
- print("Bonjour tout le monde !")
- Résultat :
- Bonjour tout le monde !

Pourquoi Python est-il intéressant pour l'IA ?

Python possède un très grand nombre de bibliothèques :

Domaine	Bibliothèques
Calcul scientifique	NumPy
Analyse de données	pandas
Visualisation	Matplotlib
Machine Learning	scikit-learn
Deep Learning	PyTorch, TensorFlow
Vision par ordinateur	OpenCV
(Natural Language Processing) NLP	Transformers
IA générative	Transformers, PyTorch
Notebooks	Jupyter

3. Installation de Python

- Pour Windows, on peut installer Python depuis le site officiel :
- https://www.python.org/
- Lors de l'installation, il est important de cocher :
- Add Python to PATH
- Puis vérifier l'installation dans l'invite de commandes:
- python –version, ou :
- py --version
- Exemple :
- Python 3.13.7

4. Les environnements de programmation

Python peut être utilisé avec plusieurs environnements

4.1 IDLE (Integrated Development and Learning Environment)

- **IDLE** est fourni avec **Python**
- On peut écrire :
- print("Bonjour")
- et exécuter le programme.

4.2 Visual Studio Code

- **VS Code** permet de programmer en **Python** avec :
  - coloration syntaxique ;
  - Auto-complétion ;
  - débogage ;
  - terminal ;
  - extensions Python

IDLE, Jupyter et VS Code

Imaginez que vous avez trois bureaux différents pour travailler avec Python :

Imaginez que vous avez trois bureaux différents pour travailler avec Python :

Outil	Image mentale	Utilisation
IDLE	📝 Petit cahier	Apprendre Python simplement
Jupyter Notebook	📓 Cahier interactif	Data Science, statistiques, graphiques
Visual Studio Code	🗂️ Atelier complet	Projets Python plus importants

1. Jupyter, c'est quoi ?

- **Jupyter** est un **environnement** qui permet de travailler de manière **interactive** avec **Python** et d'autres langages
- Son intérêt principal est qu'on peut **écrire du code**, **l'exécuter immédiatement et voir le résultat juste en dessous**
- Par exemple :

Par exemple :

- **x = 10**
- **y = 20**
- **x + y**

- **On obtient:**

- **30**
- C'est particulièrement pratique pour **Python**, **les statistiques**, **pandas**, **NumPy**, **Matplotlib**, **Machine Learning et Deep Learning**.

2. Jupyter Notebook, c'est quoi ?

Un **Jupyter Notebook** est un document interactif dans lequel on travaille.
Il est constitué de **cellules** (*cells*).

Par exemple :

Cellule 1 — texte :

Analyse des résultats de mon expérience

Cellule 2 — Python :

import numpy as np notes = [12, 15, 17, 14]
np.mean(notes)

Résultat :

14.5

2. Jupyter Notebook, c'est quoi ?

Puis une autre cellule peut contenir un graphique :

```
import matplotlib.pyplot as plt

plt.plot(notes)

plt.show()
```

Le graphique apparaît directement dans le notebook.

4.3 Jupyter Notebook

- **Jupyter** est particulièrement intéressant pour :
  - l'enseignement ;
  - les mathématiques ;
  - la data science ;
  - l'intelligence artificielle.

- Un programme peut être exécuté progressivement :
- x = 10
- y = 20
-
- x + y

- Résultat :
- 30

Jupyter Notebook vs IDLE

Comme vous êtes justement en train d'apprendre Python, cette comparaison va vous aider :

	IDLE	Jupyter Notebook
Écrire du Python	✓	✓
Exécuter du code	✓	✓
Cellules indépendantes	✗	✓
Texte + code + résultats	Limité	✓
Graphiques intégrés	Limité	✓
pandas / NumPy	✓	★★★
Machine Learning	✓	★★★
Deep Learning	✓	★★★
Très pratique pour la recherche	Moyen	★★★
Débuter en Python	★★★	★★★

Pour votre apprentissage

- Je vous conseillerais de **garder IDLE pour apprendre les bases de Python** :
- variables → listes → fichiers → fonctions, etc.
- Puis, lorsque nous arriverons à:
  - **pandas**,
  - **NumPy**,
  - **statistiques**,
  - **Matplotlib**,
  - **Machine Learning et**
  - **Deep Learning**,
- **Jupyter Notebook deviendra très intéressant pour vous**.

Premier programme Python

Créer un fichier :

- premier_programme.py

Écrire :

- print("Bienvenue dans Python")
- print("Cours d'initiation à Python")

Exécution :

- python premier_programme.py

6. Les commentaires

Commentaires

- Un commentaire commence par #.
- # Ceci est un commentaire
-
- print("Bonjour")
- Les commentaires permettent d'expliquer le programme

Exemple

- # Calcul de la moyenne de deux nombres
-
- a = 10
- b = 20
-
- moyenne = (a + b) / 2
-
- print(moyenne)

7. Les variables

- **Une variable permet de stocker une valeur.**
- nom = "Ali"
- age = 20
- note = 15.5
- **On peut afficher les valeurs :**
- print(nom)
- print(age)
- print(note)
- **Ou :**
- print("Nom :", nom)
- print("Age :", age)
- print("Note :", note)
- **Python n'oblige pas à déclarer explicitement le type de la variable.**

8. Les principaux types de données

Entier : int

- age = 20

Réel : float

- note = 15.75

Chaîne de caractères : str

- nom = "Zair"

Booléen : bool

- admis = True
- ou :
- admis = False

On peut connaître le type :

- x = 10
-
- print(type(x))
- Résultat :
- <class 'int'>

9. Les opérations arithmétiques

Python permet de réaliser facilement des calculs.

Opération	Symbole
Addition	+
Soustraction	-
Multiplication	*
Division	/
Division entière	//
Reste	%
Puissance	**

Exemple :

```
a = 10
b = 3
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
print(a % b)
print(a ** b)
```

10. Lire une donnée au clavier

La fonction input() permet de demander une information à l'utilisateur

- nom = input("Entrez votre nom : ")
-
- print("Bonjour", nom)

Exemple :

- Entrez votre nom : Ahmed
- Bonjour Ahmed

Attention

- **input()** retourne une chaîne de caractères.
- Donc :
- **age** = input("Entrez votre âge : ")
- **age** est une **chaîne**.
- Pour obtenir un **entier** :
- **age** = int(input("Entrez votre âge : "))
- Pour obtenir un **réel** :
- **note** = float(input("Entrez votre note : "))

11. Exemple : calcul de la moyenne

```
note1 = float(input("Note 1 : "))
note2 = float(input("Note 2 : "))
note3 = float(input("Note 3 : "))

moyenne = (note1 + note2 + note3) / 3

print("Moyenne =", moyenne)
```

12. Les conditions

Python utilise if, elif et else

```
age = 20

if age >= 18:
    print("Majeur")
else:
    print("Mineur")
```

Exemple avec plusieurs conditions

```
note = 15

if note >= 16:
    print("Très bien")
elif note >= 14:
    print("Bien")
elif note >= 10:
    print("Passable")
else:
    print("Échec")
```

13. Les opérateurs de comparaison

Opérateur	Signification
==	égal
!=	différent
>	supérieur
<	inférieur
>=	supérieur ou égal
<=	inférieur ou égal

Exemple :

```
age = 20
if age >= 18:
    print("Accès autorisé")
```

14. Les opérateurs logiques

Python utilise :

- and
- or
- not

Exemple :

- age = 20
- note = 15
-
- if age >= 18 and note >= 10:
- print("Condition satisfaite")

15. Les boucles

Les boucles permettent de répéter des instructions.